

Introduction to Programming with Haskell

Defining functions

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Haskell

- A programming language for describing functions

Haskell

- A programming language for describing functions
- A function description has two parts

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs
- **Definition** or **rule** for computing outputs from inputs

Haskell

- A programming language for describing functions
- A function description has two parts
- **Type** – of inputs and outputs
- **Definition** or **rule** for computing outputs from inputs
- Example function

```
sqr :: Int → Int      -- Type specification  
sqr x = x * x         -- Computation rule
```

Defining functions

- `xor` (Exclusive or)

Defining functions

- `xor` (Exclusive or)
- Input two values of type `Bool`

Defining functions

- `xor` (Exclusive or)
- Input two values of type `Bool`
- Check that exactly one of them is `True`

```
xor1 :: Bool → Bool → Bool           -- why?  
xor1 b1 b2 = (b1 && (not b2)) || ((not b1) && b2)
```

Defining functions: if-then-else

- `xor` (Exclusive or)

```
xor2 :: Bool → Bool → Bool
```

```
xor2 b1 b2 = if b1 == b2 then False else True
```

Defining functions: if-then-else

- `xor` (Exclusive or)

```
xor2 :: Bool → Bool → Bool  
xor2 b1 b2 = if b1 == b2 then False else True
```

- `if - then - else` is an **expression**, not a statement!

Defining functions: if-then-else

- **xor** (Exclusive or)

```
xor2 :: Bool → Bool → Bool  
xor2 b1 b2 = if b1 == b2 then False else True
```

- **if - then - else** is an **expression**, not a statement!
- The **then** and **else** cases define values

Defining functions: if-then-else

- `xor` (Exclusive or)

```
xor2 :: Bool → Bool → Bool  
xor2 b1 b2 = if b1 == b2 then False else True
```

- `if - then - else` is an **expression**, not a statement!
- The **then** and **else** cases define values
- Akin to `cond?expr1:expr2;` in languages like C

Defining `xor`

- `xor` can also be defined as:

```
xor3 :: Bool → Bool → Bool  
xor3 b1 b2 = if b1 /= b2 then True else False
```

Defining xor

- `xor` can also be defined as:

```
xor3 :: Bool → Bool → Bool  
xor3 b1 b2 = if b1 /= b2 then True else False
```

- Can be simplified to:

```
xor4 :: Bool → Bool → Bool  
xor4 b1 b2 = b1 /= b2
```

Defining xor

- Another version:

```
xor5 :: Bool → Bool → Bool  
xor5 b1 b2 = (/=) b1 b2
```

Defining xor

- Another version:

```
xor5 :: Bool → Bool → Bool  
xor5 b1 b2 = (/=) b1 b2
```

- Haskell operators, even infix operators, are just functions!

Defining xor

- Another version:

```
xor5 :: Bool → Bool → Bool  
xor5 b1 b2 = (/=) b1 b2
```

- Haskell operators, even infix operators, are just functions!
- They can be used like functions (prefix) by enclosing in parentheses

Defining xor

- Another version:

```
xor5 :: Bool → Bool → Bool  
xor5 b1 b2 = (/=) b1 b2
```

- Haskell operators, even infix operators, are just functions!
- They can be used like functions (prefix) by enclosing in parentheses
- $(*)\ x\ y$ is the same as $x\ *\ y$

Defining xor

- Another version:

```
xor5 :: Bool → Bool → Bool  
xor5 b1 b2 = (/=) b1 b2
```

- Haskell operators, even infix operators, are just functions!
- They can be used like functions (prefix) by enclosing in parentheses
- $(*)\ x\ y$ is the same as $x\ *\ y$
- Yet another version:

```
xor6 :: Bool → Bool → Bool  
xor6 = (/=)
```

Pattern matching

- Multiple definitions (for various patterns of input):

```
xor7 :: Bool → Bool → Bool
xor7 False False = False
xor7 False True  = True
xor7 True  False = True
xor7 True  True  = False
```

Pattern matching

- Not always feasible to list all inputs

```
xor8 :: Bool → Bool → Bool
xor8 False True  = True
xor8 True  False = True
xor8 b1    b2    = False
```

Pattern matching

- Not always feasible to list all inputs

```
xor8 :: Bool → Bool → Bool
xor8 False True  = True
xor8 True  False = True
xor8 b1    b2    = False
```

- Use first definition that matches, from top to bottom

Pattern matching

- Not always feasible to list all inputs

```
xor8 :: Bool → Bool → Bool
xor8 False True  = True
xor8 True  False = True
xor8 b1    b2    = False
```

- Use first definition that matches, from top to bottom
 - `xor8 False True` matches first definition

Pattern matching

- Not always feasible to list all inputs

```
xor8 :: Bool → Bool → Bool
xor8 False True  = True
xor8 True  False = True
xor8 b1    b2    = False
```

- Use first definition that matches, from top to bottom
 - `xor8 False True` matches first definition
 - `xor8 True True` matches third definition

Pattern matching

- When does a function call match a definition?

Pattern matching

- When does a function call match a definition?
- If argument in the definition is a constant

Pattern matching

- When does a function call match a definition?
- If argument in the definition is a constant
 - value supplied in the function call must be the same constant

Pattern matching

- When does a function call match a definition?
- If argument in the definition is a constant
 - value supplied in the function call must be the same constant
- If argument in the definition is a variable

Pattern matching

- When does a function call match a definition?
- If argument in the definition is a constant
 - value supplied in the function call must be the same constant
- If argument in the definition is a variable
 - any value supplied in the function call matches, and is substituted for the variable (**the usual case**)

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

- `xor9 False True` and `xor9 False False` match first definition

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

- `xor9 False True` and `xor9 False False` match first definition
- `xor9 True False` matches second definition

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

- `xor9 False True` and `xor9 False False` match first definition
- `xor9 True False` matches second definition
- `xor9 True True` matches third definition

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

- `xor9 False True` and `xor9 False False` match first definition
- `xor9 True False` matches second definition
- `xor9 True True` matches third definition
- The argument `b` is used on the RHS in the first two definitions

Pattern matching

- Can mix constants and variables in a definition

```
xor9 :: Bool → Bool → Bool
xor9 False b      = b
xor9 b      False = b
xor9 b1     b2     = False
```

- `xor9 False True` and `xor9 False False` match first definition
- `xor9 True False` matches second definition
- `xor9 True True` matches third definition
- The argument `b` is used on the RHS in the first two definitions
- `b1` and `b2` are ignored on the RHS in the third definition

Pattern matching

- A better version:

```
xor10 :: Bool → Bool → Bool  
xor10 False b    = b  
xor10 True  b     = not b
```

Pattern matching

- A better version:

```
xor10 :: Bool → Bool → Bool  
xor10 False b    = b  
xor10 True  b    = not b
```

- `b` is used on the RHS in both definitions

Pattern matching: wildcards

- Another example:

```
xor11 :: Bool → Bool → Bool
xor11 False True  = True
xor11 True  False = True
xor11 _     _     = False
```

Pattern matching: wildcards

- Another example:

```
xor11 :: Bool → Bool → Bool
xor11 False True  = True
xor11 True  False = True
xor11 _     _     = False
```

- The symbol `_` denotes a “don’t care” argument

Pattern matching: wildcards

- Another example:

```
xor11 :: Bool → Bool → Bool
xor11 False True  = True
xor11 True  False = True
xor11 _     _     = False
```

- The symbol `_` denotes a “don’t care” argument
- Any value matches this pattern

Pattern matching: wildcards

- Another example:

```
xor11 :: Bool → Bool → Bool
xor11 False True  = True
xor11 True  False = True
xor11 _     _     = False
```

- The symbol `_` denotes a “don’t care” argument
- Any value matches this pattern
- The value is not captured, cannot be reused